SENTIMENT ANALYSIS SERVICE

Engineering Handbook and Organizational Ownership Report

Internal production-readiness and handover document

Tech Lead	Le Quang Tuyen
ML Engineering Lead	Duong Binh An
AI Engineering Lead	Do Quoc Trung
Solution Engineering Lead	Duong Hong Quan
Backend Engineering Lead	Pham Duc Long
UI/UX Engineering Lead	Nguyen Le Hong Nhi

Source-of-truth rule: this handbook trusts implementation first, then runtime configuration, then tests, then legacy reports. Where documents and code disagree, the operational truth is the code path currently executed by the repository.

Contents

1 Executive Assessment

1.1 Business Purpose

The product turns unstructured customer feedback into structured operational signals: sentiment, aspect-level sentiment, sarcasm cues, and explanation artifacts. The intended business use is review analysis, service-quality monitoring, support triage, and explainable AI demonstration.

1.2 Product Purpose

The repository contains two products sharing one codebase:

- an online inference service exposed through Angular and FastAPI
- an offline ML system for data preparation, finetuning, evaluation, MLflow tracking, and ONNX packaging

1.3 Technical Purpose

The system is designed to prove that the team can operate a full ML delivery chain:

- DVC-managed data flow
- Hugging Face and PEFT training
- ONNX runtime serving
- Prometheus and Grafana observability
- containerized deployment through Docker Compose

1.4 Core Value Proposition

The main value is not raw model novelty. It is the operational packaging of sentiment inference into a service with explainability, batch processing, monitoring, and artifact lineage.

1.5 Maturity Assessment

Rating	Assessment
Prototype	Exceeded. The repo has API contracts, tests, monitoring, Docker packaging, DVC, MLflow, and ONNX export.
MVP	Exceeded. Users can run single prediction, explanation, health, metrics, and CSV batch flows.
Advanced MVP	**Current state**. The service demonstrates end-to-end product behavior and a non-trivial ML lifecycle.
Pre-Production	Partial only. Major production gaps remain: no authentication, permissive CORS, in-memory evaluation state, mocked batch status, and no real job orchestration.
Production Ready	Not approved. The deployment is useful for demos, internal validation, and controlled pilots, but not enterprise production.

1.6 Why it is not production ready

- `src/main.py` exposes all inference and evaluation endpoints without auth.
- `/batch_status/{job_id}` returns a mock response and is not backed by a queue or durable store.
- the frontend language selector emits a language but `SentimentAnalysisService.predict()` drops it and only sends `{text}`.
- Docker Compose ships Grafana with default admin password `admin`.
- health checks only prove model load, not downstream dependencies such as MLflow, Prometheus scrape health, or artifact integrity.

2 System Mental Model

2.1 Business View

```
flowchart LR
  Customer[Customer or Analyst]
  UI[Angular Chat UI]
  API[FastAPI Service]
  Insight[Sentiment and Aspect Insights]
  Ops[Monitoring and Reports]
  Improve[Retraining and Export]
  Customer --> UI --> API --> Insight --> Ops --> Improve
```

2.2 Product View

```
flowchart TB
  A[Single Predict]
  B[Explain SHAP]
  C[Batch CSV Predict]
  D[Background Evaluate]
  E[Dashboard and Metrics]
  UI[Frontend]
  UI --> A
  UI --> B
  UI --> C
  API[Backend]
  A --> API
  B --> API
  C --> API
  D --> API
  API --> E
```

2.3 Engineering View

```
flowchart LR
```

```
  FE[app/sentiment-analysis-chatbot]
  BE[src/main.py]
  MODEL[src/model]
  DATA[src/data]
  TRAIN[src/scripts + src/training]
  OBS[infra + src/monitoring]
  FE --> BE --> MODEL
  DATA --> TRAIN
  TRAIN --> MODEL
  BE --> OBS
```

2.4 Infrastructure View

```
flowchart LR
  Browser --> Nginx[Frontend container :80]
  Nginx --> FastAPI[fastapi_app :8000]
  FastAPI --> ML[Model runtime in-process]
  FastAPI --> Metrics[/metrics]
  Metrics --> Prometheus[:9091]
  Prometheus --> Grafana[:3000]
  FastAPI --> Eval[/evaluate]
  Eval --> MLflow[:5005]
```

2.5 AI/ML View

```
flowchart LR
  Raw[data/raw]
  Prep[src/data/pipeline.py]
  Validate[src/data/validators.py]
  Train[src/scripts/run_finetuning.py]
  Eval[src/scripts/evaluate_finetuned.py and src/model/evaluate.py]
  Export[src/scripts/export_onnx.py]
  Serve[src/model/baseline.py]
  Raw --> Prep --> Validate --> Train --> Eval --> Export --> Serve
```

3 End-to-End Business Flow

Step	Business Objective	Technical Implementation	Owner
Customer	Submit text feedback for analysis	User interacts with Angular chat input or CSV upload in `app/sentiment-analysis-chatbot/src/app/components`.	UI/UX Lead
UI	Collect text, voice transcription, or CSV file	`ChatInputComponent` emits text events; `BatchUploadComponent` uploads CSV through multipart form.	UI/UX Lead
API	Validate request and route to the right execution path	`src/main.py` validates with Pydantic schemas in `contracts/schemas.py`.	Backend Lead

Step	Business Objective	Technical Implementation	Owner
Inference	Produce sentiment, aspects, sarcasm, or SHAP explanations	`BaselineModelInference` dispatches to HF or ONNX runtime and ABSA pipeline.	AI Lead
Result	Return structured response to UI or API caller	`PredictResponse`, `ExplainResponse`, and batch response schemas serialize outputs.	Backend Lead
Monitoring	Capture request count and latency	`src/monitoring/metrics.py` middleware and Prometheus scrape endpoint.	Solution Lead
Feedback	Review metrics and evaluation artifacts	Grafana dashboards, MLflow runs, JSON reports under `reports/` and `data/reports/`.	Solution Lead / ML Lead
Retraining	Improve adapters with new or revised datasets	`src/scripts/run_finetuning.py`, task configs, class weights, and LoRA adapters.	ML Lead
Deployment	Package updated runtime artifacts for serving	Dockerfiles, Docker Compose, ONNX export scripts, and mounted model directories.	Solution Lead / Backend Lead

4 Architecture Deep Dive

4.1 Frontend

Responsibilities: message composition, voice capture, history persistence, explain trigger, batch upload, theme control.
Dependencies: Angular, HttpClient, local storage, backend `/api/*` routes via Nginx.
Risks: contract drift, hidden language-selector bug, limited frontend test depth.
Ownership: UI/UX Lead primary, Backend Lead secondary for API coupling.

4.2 Backend

Responsibilities: startup lifecycle, dependency injection, schema validation, exception mapping, health, metrics, evaluate trigger, batch orchestration.
Dependencies: FastAPI, pandas, model interface, monitoring middleware, background tasks.
Risks: single-process bottleneck, synchronous expensive paths, no auth, in-memory state.
Ownership: Backend Lead primary, Solution Lead secondary.

4.3 Inference Layer

Responsibilities: sentiment scoring, sarcasm adapter path, ABSA extraction, SHAP explanation, ONNX session loading, language validation.
Dependencies: Hugging Face, PEFT, SHAP, onnxruntime, tokenizer artifacts, exported models.
Risks: startup latency, SHAP cost, ABSA latency, artifact mismatch, HF model download dependence.
Ownership: AI Lead primary, ML Lead secondary.

4.4 Training Layer

Responsibilities: dataset loading, deduplication, split strategy, imbalance handling, LoRA config, trainer setup, MLflow logging.
Dependencies: Datasets, Transformers, PEFT, sklearn, task configs, params.
Risks: data skew, limited reproducibility outside configured data files, implicit dependence on raw file availability.
Ownership: ML Lead primary, AI Lead secondary.

4.5 Monitoring Layer

Responsibilities: request metrics, dashboard provisioning, alert rule definitions.
Dependencies: Prometheus, Grafana, middleware instrumentation.
Risks: no tracing, limited model-level business metrics, alerts cover only a narrow failure set.
Ownership: Solution Lead primary, Backend Lead secondary.

4.6 Deployment Layer

Responsibilities: image build, service topology, ports, resource limits, network and mounted storage.
Dependencies: Dockerfile, Dockerfile.train, docker-compose, model and data directories.
Risks: no orchestration beyond Compose, secret handling through build args, default credentials, no blue/green or rollback automation.
Ownership: Solution Lead primary, Backend Lead secondary.

5 Repository Ownership Map

Folder	Purpose	Primary	Secondary	Business Impact	Modification Risk	Knowledge Risk
`app/`	Angular customer-facing UI	UI/UX	Backend	High	Medium	Medium
`contracts/`	Shared API and model contracts	Backend	AI	High	High	Medium
`data/`	Raw, processed, eval, and report artifacts	ML	AI	High	High	High
`docs/`	Legacy reports and handbook outputs	Tech Lead	All leads	Medium	Low	Low
`infra/`	Prometheus and Grafana provisioning	Solution	Backend	High	Medium	Medium
`mlruns/`	MLflow local experiment store	ML	Solution	Medium	Low	Medium
`models/`	Adapters and ONNX artifacts	AI	ML	High	High	High
`notebooks/`	Colab/manual pipeline execution	ML	AI	Medium	Medium	High

Folder	Purpose	Primary	Secondary	Business Impact	Modification Risk	Knowledge Risk
`reports/`	Evaluation and benchmark outputs	ML	AI	Medium	Low	Medium
`src/data/`	Download, preprocess, validate flows	ML	AI	High	High	Medium
`src/model/`	Runtime inference and export logic	AI	ML	High	High	High
`src/monitoring/`	Prometheus instrumentation	Solution	Backend	Medium	Low	Low
`src/scripts/`	Operational CLI entrypoints	ML	AI	High	Medium	Medium
`src/training/`	Reusable training internals	ML	AI	High	Medium	Medium
`tests/`	Regression safety net	Tech Lead	Owning team	High	Medium	Medium
`.github/`	CI behavior	Backend	Tech Lead	High	Medium	Medium

5.1 Folder Ownership Matrix

Lead	Core folders	Review triggers	Single-point-of-failure risk	Backup owner	Notes
Tech Lead	`docs/`, `tests/`, cross-cutting changes	architecture, security, readiness	Medium	Solution Lead	final approver for major changes
ML Lead	`src/data/`, `src/training/`, `data/`	datasets, training params, evaluation metrics	High	AI Lead	owns data quality gates
AI Lead	`src/model/`, `models/`	inference mode, ABSA, SHAP, ONNX	High	ML Lead	highest artifact coupling
Solution Lead	`infra/`, `docker-compose.yml`, deployment docs	observability and runtime topology	Medium	Backend Lead	owns end-to-end ops fit
Backend Lead	`src/main.py`, `contracts/`, CI	API routes, schemas, error handling	Medium	Solution Lead	owns request semantics
UI/UX Lead	`app/`	user flows, upload UX, explain UX	Medium	Backend Lead	tightly coupled to response schema

6 Runtime Execution Trace

6.1 Execution Tables

Flow	File / Class	Function	Input	Output	Exception Path	Owner
Startup	`src/main.py` / module	`lifespan()`	env `MODEL_MODE`	initialized `ml_model`	`ModelError` on load failure	Backend + AI
Health	`src/main.py` / API	`health_check()`	HTTP GET	`HealthResponse`	503 if model not ready through dependency	Backend
Predict	`src/main.py` / API	`predict()`	`PredictRequest`	`PredictResponse`	400 unsupported language, 500 model error	Backend
Predict core	`src/model/baseline.py` / `BaselineModelInference`	`predict_single()`	text, lang	`PredictionResult`	`UnsupportedLanguage`, `ModelError`	AI
Explain	`src/main.py` / API	`explain()`	`ExplainRequest`	`ExplainResponse`	400 unsupported language, 500 explainer failure	Backend + AI
Batch Predict	`src/main.py` / API	`batch_predict()`	uploaded CSV	`BatchPredictResponse`	400 invalid CSV or missing column, 500 batch failure	Backend
Batch core	`src/model/baseline.py` / `BaselineModelInference`	`predict_batch()`	list of texts	list of `PredictionResult`	`ValueError` invalid batch size	AI
Evaluate trigger	`src/main.py` / API	`run_evaluate()`	POST no body	status JSON	409 already running; background `last_error` on failure	Backend + ML
Evaluate status	`src/main.py` / API	`evaluate_status()`	HTTP GET	in-memory state JSON	stale state after restart	Backend
Offline eval	`src/model/evaluate.py`	`evaluate_on_dataset()`	processed test frame	metrics dict	returns error payload when split empty	ML + AI

6.2 Sequence Diagrams

Startup

```
sequenceDiagram
  participant U as Uvicorn/FastAPI
  participant A as src.main.lifespan
  participant C as ModelConfig
  participant M as BaselineModelInference
  participant O as OnnxInferenceSession or HF model
  U->>A: app startup
  A->>C: read MODEL_MODE
  A->>M: instantiate
  M->>O: load tokenizer/model/session
  A->>M: preload()
  M->>M: warm ABSA pipeline
  A-->>U: app ready
```

Predict

```
sequenceDiagram
  participant User
  participant FE as Angular UI
  participant API as FastAPI /predict
  participant LD as LanguageDetector
  participant MI as BaselineModelInference
  participant ABSA as Zero-shot pipeline
  User->>FE: enter text
  FE->>API: POST /api/predict {text}
  API->>LD: resolve_request_language()
  API->>MI: predict_single(text, lang)
  MI->>MI: sentiment logits
  MI->>ABSA: optional aspect extraction
  MI-->>API: PredictionResult
  API-->>FE: PredictResponse
  FE-->>User: render sentiment and aspects
```

Explain

```
sequenceDiagram
  participant FE as Angular UI
  participant API as FastAPI /explain
  participant MI as BaselineModelInference
  participant SH as SHAP Explainer
  FE->>API: POST /api/explain {text}
  API->>MI: get_shap_explanation()
  MI->>SH: build explainer and score text
  SH-->>MI: token values
  MI-->>API: SHAPResult
  API-->>FE: ExplainResponse
```

Batch Predict

```
sequenceDiagram
  participant User
  participant FE as BatchUploadComponent
  participant API as FastAPI /batch_predict
  participant PD as pandas.read_csv
  participant MI as BaselineModelInference
  User->>FE: upload CSV
  FE->>API: multipart file
  API->>PD: parse CSV bytes
  API->>MI: predict_batch(valid_texts, skip_absa=true)
  MI-->>API: prediction list
  API-->>FE: batch response with row results
```

Evaluate

```
sequenceDiagram
  participant Caller
  participant API as FastAPI /evaluate
  participant BG as Background task
  participant Eval as src.model.evaluate
  participant MLF as MLflow
  Caller->>API: POST /evaluate
  API-->>Caller: started
  API->>BG: add task
  BG->>Eval: evaluate_on_dataset()
  Eval->>MLF: log_to_mlflow()
  BG-->>API: update in-memory state
```

7 Team Handbooks

7.1 7.1 Tech Lead Handbook

Owner: Le Quang Tuyen
Responsibilities: architecture direction, code-review bar, readiness approvals, cross-team conflict resolution.
Systems owned: whole-system architecture, standards, release quality gates.
Files owned: handbook, test strategy, cross-cutting config changes.
Operational duties: production-readiness review, incident commander for Sev-1.
Release duties: approve contract changes, security exceptions, and go-live checklist.
Risks: architecture drift and unclear ownership boundaries.
KPIs: escaped-defect rate, release predictability, review cycle time, readiness score trend.

7.2 7.2 ML Engineering Handbook

Owner: Duong Binh An
Responsibilities: datasets, preprocessing, validation thresholds, finetuning, evaluation design, MLflow.
Systems owned: `src/data/`, `src/training/`, training scripts, `data/`, `reports/`.

Files owned: `src/scripts/run_finetuning.py`, `src/data/validators.py`, `params.yaml`, `dvc.yaml`.
Operational duties: maintain dataset integrity and evaluation reproducibility.
Release duties: validate metrics before artifact promotion.
Risks: knowledge silo around data lineage and imbalance logic.
KPIs: macro F1, per-language F1, reproducibility success, validation pass rate.

7.3 7.3 AI Engineering Handbook

Owner: Do Quoc Trung
Responsibilities: inference behavior, ONNX runtime, ABSA, sarcasm flag, SHAP, language detection.
Systems owned: `src/model/`, `models/`.
Files owned: `baseline.py`, `onnx_inference.py`, `onnx_exporter.py`, `language_detector.py`.
Operational duties: model load diagnostics, runtime latency tuning, artifact compatibility checks.
Release duties: sign off on model artifact integrity and inference regression tests.
Risks: artifact mismatch, SHAP cost explosions, hidden provider issues.
KPIs: p95 latency, model load success, inference error rate, benchmark stability.

7.4 7.4 Solution Engineering Handbook

Owner: Duong Hong Quan
Responsibilities: end-to-end integration, monitoring stack, deployment topology, service packaging.
Systems owned: `infra/`, Compose topology, release runbooks.
Files owned: `docker-compose.yml`, `Dockerfile`, `Dockerfile.train`, Prometheus and Grafana configs.
Operational duties: maintain observability and environment consistency.
Release duties: verify service health, ports, mounted volumes, and dashboards.
Risks: dependency drift, insecure defaults, manual deployment steps.
KPIs: deployment success rate, monitoring freshness, incident MTTR.

7.5 7.5 Backend Engineering Handbook

Owner: Pham Duc Long
Responsibilities: API semantics, request validation, exception handling, evaluate and batch endpoints.
Systems owned: `src/main.py`, `contracts/`, backend CI and tests.
Files owned: `src/main.py`, `contracts/schemas.py`, `contracts/errors.py`.
Operational duties: endpoint health, error triage, backward compatibility.
Release duties: maintain OpenAPI correctness and contract stability.
Risks: route drift, in-memory state, unbounded expensive calls.
KPIs: 5xx rate, schema-change defect rate, endpoint coverage.

7.6 7.6 UI/UX Engineering Handbook

Owner: Nguyen Le Hong Nhi

Responsibilities: user journey, interaction quality, input capture, explain visualization, batch results UX.

Systems owned: Angular application and related styles.

Files owned: `app.component.ts`, components, services, models.

Operational duties: validate client behavior against backend contract.

Release duties: ensure visible features map to real backend support.

Risks: hidden contract mismatch, low automated coverage, local storage state issues.

KPIs: task completion rate, UI defect rate, batch UX success, explain usage.

8 RACI Matrix

Workstream	Tech	ML	AI	Solution	Backend	UI/UX
Feature development	A	C	C	C	R	R
Model training	C	A/R	C	I	I	I
Model deployment	A	C	R	R	C	I
API changes	A	I	C	C	R	C
Production release	A	C	C	R	R	C
Incident management	A	C	R	R	R	I
Monitoring	C	I	C	A/R	R	I
Security	A	I	C	R	R	C
Architecture changes	A/R	C	C	C	C	C

9 Data and Model Lineage

```
flowchart LR
  Raw[data/raw/*.csv]
  Validation1[src.data.downloader.validate_raw_schema]
  Prep[src.data.pipeline.PreprocessingPipeline.run]
  Validation2[src.data.validators.DataQualityValidator.validate]
  Train[src.scripts.run_finetuning.train]
  Eval[src.model.evaluate or src.scripts.evaluate_finetuned]
  Track[MLflow runs]
  Export[src.scripts.export_onnx]
  Runtime[src.main -> BaselineModelInference]
  Predict[PredictResponse]
  Raw --> Validation1 --> Prep --> Validation2 --> Train --> Eval --> Track
  --> Export --> Runtime --> Predict
```

Stage	Implementation	Owner
Raw data	`src/data/downloader.py` writes `data/raw/*`	ML Lead
Validation	raw-schema checks and processed-data validator	ML Lead
Training	`src/scripts/run_finetuning.py` + `src/training/*`	ML Lead
Evaluation	`src/model/evaluate.py` and finetuned evaluation scripts	ML Lead + AI Lead
MLflow	tracking URI resolution and logged metrics/artifacts	ML Lead
ONNX export	`src/model/onnx_exporter.py` via CLI script	AI Lead
Runtime load	`BaselineModelInference._load_model()`	AI Lead
Prediction	`predict_single()` and `predict_batch()`	AI Lead

10 Security Review

Finding	Severity	Owner	Remediation Plan
No authentication or authorization on inference and evaluation endpoints	Critical	Backend + Solution	add API auth, role-based access for evaluation/admin flows, and gateway-level enforcement
Permissive CORS `allow_origins=["*"]`	High	Backend	restrict origins by environment and disable credentials with wildcard patterns
Grafana default password in Compose	High	Solution	move admin secret to environment secret store and require first-run rotation
Build args for DagsHub credentials	High	Solution	stop baking secrets into build context; use runtime secrets or CI secret mount only
CSV upload without content scanning or size control beyond row cap	Medium	Backend	enforce content-type, file size limit, and structured upload validation
Model artifact trust boundary is weak	Medium	AI + ML	add checksum verification and signed artifact promotion
Background evaluation exposed as public API	High	Backend	place behind admin auth or remove from public runtime surface
No rate limiting	High	Backend + Solution	add reverse-proxy and app-level controls for expensive routes

11 Operations Handbook

Scenario	Symptoms	Likely Root Cause	Debug Steps	Resolution	Owner
API down	`/health` fails, UI overlay never clears	model load failure, container crash, bad artifact path	inspect backend logs, confirm `MODEL_MODE`, verify model files, call container directly	restore valid artifacts or roll back image	Backend + AI
Model failure	500 on predict or explain	tokenizer/session mismatch, corrupted adapter or ONNX file	run targeted API call, inspect `ModelError`, test local load in shell	redeploy last known-good model artifact	AI

Scenario	Symptoms	Likely Root Cause	Debug Steps	Resolution	Owner
Latency spike	p95 request latency rises, Grafana alert fires	SHAP use, ABSA overhead, CPU saturation	compare `/predict` vs `/explain`, inspect CPU limits, review batch load	disable heavy paths or scale runtime resources	AI + Solution
Monitoring failure	Grafana empty, Prometheus scrape errors	broken metrics endpoint or config drift	query `/metrics`, inspect Prometheus targets, check provisioning files	fix scrape target or dashboard datasource	Solution
Batch failure	400 or 500 on CSV upload	malformed CSV, missing `text` column, runtime error	reproduce with sample CSV, inspect parser error, check row content	fix client file or harden parser path	Backend
Deployment failure	one or more Compose services unhealthy	secret/config drift, missing volume data, bad image build	inspect `docker-compose` logs and env injection	restore config, rebuild, or revert	Solution

12 Technical Debt Register

Level	Debt Item	Impact	Risk	Owner	Fix Cost	Priority
Critical	Public unauthenticated inference and admin-like evaluation surface	enterprise blocking	security and abuse	Backend	Medium	P0
High	Mocked `/batch_status` endpoint with no durable jobs	misleading product semantics	operational confusion	Backend	Medium	P1
High	Frontend language selector not propagated to backend	wrong user expectation and false multilingual UX	correctness drift	UI/UX	Low	P1
High	Default Grafana credentials and broad CORS	security posture failure	compromise risk	Solution	Low	P1
Medium	In-memory evaluation state lost on restart	weak operational continuity	status inconsistency	Backend	Low	P2
Medium	SHAP created on demand without caching or guardrails	latency and memory spikes	runtime instability	AI	Medium	P2
Medium	Docs claim broader readiness than implementation proves	handover confusion	decision error	Tech Lead	Low	P2
Low	Frontend automated coverage is thin	slower UI regression detection	maintainability	UI/UX	Medium	P3

13 Organizational Risk Review

- **Knowledge silos:** inference internals and artifact promotion are concentrated in AI/ML ownership.
- **Single points of failure:** `src/model/baseline.py`, `src/scripts/run_finetuning.py`, and `docker-compose.yml` are each central files with broad blast radius.

- **Bus factor:** effectively low for ONNX export, imbalance handling, and observability provisioning.
- **Cross-team dependencies:** label-set changes require ML, AI, backend, contracts, UI, and tests to move together.
- **Ownership gaps:** no explicit security owner and no explicit release manager beyond implied lead roles.

Mitigations

- require backup reviewer on every model-runtime and deployment change
- document artifact promotion and rollback as a formal runbook
- create ownership CODEOWNERS by folder
- add release checklist with named accountable approver
- schedule cross-training for AI/ML and Solution/Backend boundaries

14 Production Readiness Review

Area	Score	Status	Reason
Architecture	74	Amber	good layering, but single-process runtime and mocked async semantics
Security	32	Red	no auth, permissive CORS, default credentials, public eval endpoint
Testing	76	Amber	broad Python test coverage, weak frontend depth, limited system tests
Monitoring	71	Amber	Prometheus and Grafana exist, but tracing and business SLIs are missing
Performance	63	Amber	ONNX path helps, but SHAP and ABSA remain expensive and unbounded
Reliability	58	Amber	health exists, but no durable queue/state, no rollback automation
ML lifecycle	79	Amber	DVC, MLflow, training, evaluation, export are well represented
Deployment	60	Amber	Compose works for controlled environments, not enterprise operations
Operations	55	Amber	basic runbooks possible, but automation and security are thin
Ownership	70	Amber	roles are clear enough after this handbook, but still person-heavy
Overall	**64**	**Amber**	advanced MVP, not approved for enterprise production

15 Roadmap

15.1 3-Month Roadmap

- add authentication, rate limiting, and environment-specific CORS. Owner: Backend + Solution. Priority: P0. Business value: unblock secure pilot use. Technical value: closes major risk surface.

- replace mocked batch status with durable job tracking. Owner: Backend. Priority: P1. Risk reduction: removes false async contract.
- fix frontend language propagation and align supported-language UX. Owner: UI/UX + Backend. Priority: P1. Business value: trust in multilingual story.

15.2 6-Month Roadmap

- add artifact registry and checksum validation for model promotion. Owner: AI + ML. Priority: P1.
- add distributed tracing and richer model SLIs. Owner: Solution. Priority: P2.
- separate admin evaluation flows from public inference deployment. Owner: Backend + Solution. Priority: P1.

15.3 12-Month Roadmap

- move from Compose-only deployment to managed environment with secure secrets and autoscaling. Owner: Solution. Priority: P1.
- formalize model registry, approval gates, and rollback policy. Owner: Tech Lead + ML + AI. Priority: P1.
- expand multilingual support only after true data, evaluation, and UI parity. Owner: ML + AI + UI/UX. Priority: P2.

16 Engineer Onboarding Guide

Milestone	Expected onboarding outcomes by role
Day 1	Tech Lead: review architecture and debt register. ML: read `src/data/` and `params.yaml`. AI: read `src/model/`. Backend: read `src/main.py` and `contracts/`. UI/UX: run Angular app and inspect service/component flow. Solution: read Compose and infra configs.
Day 3	Run tests relevant to the role. Trace one live request end to end. Review one discrepancy between docs and code.
Day 7	Make a small supervised change in owned area and update tests. Learn rollback path for that subsystem.
Day 14	Execute one full workflow: training or inference or deployment, depending on role. Present risks and dependencies to the team.
Day 30	Be able to own one production-style issue in the area, explain runtime and artifact lineage, and review another engineer's change in that domain.

17 CTO Review

17.1 Would I approve production launch?

No, not for open or enterprise production. I would approve internal demo, teaching use, and a tightly controlled pilot after basic security hardening.

17.2 Would I approve enterprise customers?

No. The current security and operational controls are below enterprise minimums.

17.3 Would I approve scaling?

Not yet. The architecture can scale conceptually, but the current deployment and control plane are not ready for it.

17.4 What concerns me most?

The biggest concern is false confidence: the repository looks production-shaped, but several critical controls are still absent or mocked. That gap is more dangerous than an obviously early-stage system.

17.5 What investment should happen next?

Invest first in platform hardening: authentication, rate limiting, artifact governance, and durable batch execution. After that, invest in multilingual correctness and traceable model operations.

Implementation Discrepancies Appendix

- README health example says supported languages are only `["en"]`; `ModelConfig.supported_languages` is actually `("en","vi")`. Operational impact: docs understate backend capability, but frontend does not safely expose it.
- README presents batch processing as async job style; implementation of `/batch_predict` is synchronous request-response, while `/batch_status` is mocked. Operational impact: clients may assume durability and polling behavior that do not exist.
- frontend exposes many language choices in `ChatInputComponent`, but backend only supports `en` and `vi`; the service layer also drops explicit language on predict. Operational impact: user-visible multilingual affordance exceeds real supported runtime.
- Compose exposes MLflow on host port `5005` while service-internal tracking uses `http://mlflow:5000`; params default to `http://localhost:5000`. Operational impact: local CLI and container runtime use different connection assumptions.